

Tarea No. 3

María Fernanda Orellana Morales, 202603539
Escuela de Mecánica Eléctrica, Facultad de Ingeniería
Universidad de San Carlos de Guatemala

Resumen—En esta tarea se realizaron diferentes programas utilizando el lenguaje de programación C, con el propósito de practicar conceptos básicos de programación. Primero, se trabajó con el tipo de dato booleano, utilizando los valores true y false. Luego, se desarrolló un programa para declarar y mostrar variables de diferentes tipos de datos, como enteros, caracteres y números decimales, aplicando los formatos correspondientes. Finalmente, se elaboró un programa que solicita dos números al usuario y realiza operaciones de suma, resta, multiplicación y división. También se implementó una condición para evitar la división entre cero y mostrar un mensaje de error cuando esta situación ocurre. Estos ejercicios permitieron reforzar el uso de variables, tipos de datos, entrada y salida de información, operadores aritméticos y estructuras condicionales en C.

A. Objetivos

Desarrollar programas básicos en lenguaje C para aplicar el uso de variables, tipos de datos, operadores y estructuras condicionales en la resolución de problemas sencillos de programación.

Objetivos Específicos:

1. Identificar el uso de diferentes tipos de datos y variables dentro de un programa escrito en lenguaje C.
2. Aplicar valores booleanos y formatos de salida para representar y mostrar información correctamente.
3. Utilizar operadores aritméticos para realizar operaciones de suma, resta, multiplicación y división con datos ingresados por el usuario.
4. Implementar una estructura condicional que permita controlar situaciones específicas, como evitar una división entre cero.

B. Marco Teórico

- **Lenguaje en Programación C:** C es un lenguaje de programación utilizado para desarrollar diferentes tipos de aplicaciones y programas. Se caracteriza por permitir trabajar con variables, tipos de datos, operadores, funciones y estructuras de control. En esta práctica se utilizó C para elaborar programas sencillos que permitieran aplicar conceptos fundamentales de programación.
- **Variables y Tipos de Datos:** Las variables son espacios de memoria utilizados para almacenar información que puede ser utilizada durante la ejecución de un programa. En C existen diferentes tipos de datos, entre ellos int para números enteros, float y double para números decimales, char para caracteres y bool para valores lógicos. La elección del tipo de dato depende de la información que se necesite

almacenar.

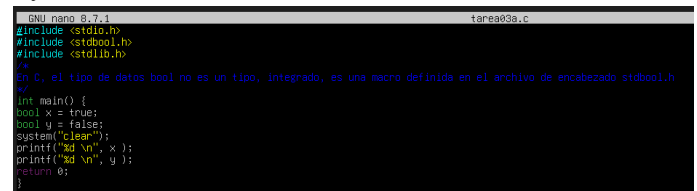
- **Valores Booleanos:** Los valores booleanos representan dos estados posibles: verdadero (true) y falso (false). En C, estos valores pueden utilizarse para representar condiciones lógicas y tomar decisiones dentro de un programa.
- **Estructuras Condicionales:** Las estructuras condicionales permiten que un programa tome diferentes acciones dependiendo de si se cumple o no una determinada condición. La estructura if permite ejecutar un bloque de instrucciones cuando una condición es verdadera. En los programas realizados se utilizó para controlar situaciones específicas, como evitar una división entre cero.
- **Entrada y Salida de Datos:** En C, las funciones printf() y scanf() permiten mostrar información en pantalla y recibir datos ingresados por el usuario, respectivamente. Estas funciones facilitan la interacción entre el programa y la persona que lo utiliza. Además, printf() utiliza especificadores de formato para mostrar correctamente los diferentes tipos de datos.
- **Compilador:** Un compilador es una herramienta que traduce el código fuente escrito en un lenguaje de programación a un lenguaje que pueda ser interpretado y ejecutado por la computadora. Para ejecutar los programas desarrollados en C, es necesario compilar el código y posteriormente ejecutar el programa generado.

C. Marco Práctico

Para la elaboración de la práctica se desarrollaron tres programas utilizando el lenguaje de programación C. Los ejercicios fueron realizados de manera progresiva, comenzando con el uso de valores booleanos, seguido por la declaración de diferentes tipos de datos y, finalmente, la creación de un programa para realizar operaciones aritméticas.

1. Creación del Primer Programa

Inicialmente, se creó un archivo con extensión .c para trabajar con valores booleanos. Para utilizar este tipo de dato se incluyó la biblioteca stdbool.h.



```
GNU nano 2.9.3 tarea03a.c
#include <stdbool.h>
#include <stdlib.h>

int main() {
    bool x = true;
    bool y = false;
    system("clear");
    printf("x: %d\n", x);
    printf("y: %d\n", y);
    return 0;
}
```

Fig. 1. Creación del Primer Programa

2. Declaración de Diferentes Tipos de Datos

Posteriormente, se elaboró un segundo programa con el objetivo de utilizar diferentes tipos de datos. Para ello se declararon variables de tipo entero, carácter y decimal, asignándoles diferentes valores.

```
GNU nano 8.7.1 tareab3b.c
#include <stdio.h>
#include <stdbool.h>
#include <stdlib.h>

En C, el tipo de datos bool no es un tipo integrado, es una macro definida en el archivo de encabezado stdbool.h

int main() {
    int i = 11;
    int j = 12;
    char c = 'A';
    float n=0.791512;
    system("clear");
    printf("x %3d %c %3f\n",i,j,c,n);
    return 0;
}
```

Fig. 2.Declaración de Diferentes Tipos de Datos

3. Ingreso de datos y operaciones aritméticas

Finalmente, se desarrolló un programa que permite al usuario ingresar dos números. Estos valores se almacenan en variables de tipo double y posteriormente se utilizan para realizar operaciones aritméticas.

```
GNU nano 8.7.1
#include <stdio.h>
int main() {
    double num1, num2;

    printf("Ingrese el primer numero: no un caracter. ");
    scanf("%lf", &num1);

    printf("Ingrese el segundo numero: no un caracter. ");
    scanf("%lf", &num2);

    double suma = num1 + num2;
    double resta = num1 - num2;
    double multiplicacion = num1 * num2;

    printf("\nResultados: \n");
    printf("Suma: %.4lf\n", suma);
    printf("Resta: %.4lf\n", resta);
    printf("Multiplicacion: %.4lf\n", multiplicacion);

    if (num2 == 0) {
        printf("Error: No se puede dividir entre cero. \n");
    } else { double division = num1 / num2;
        printf("Division: %.4lf\n", division);
    }

    return 0;
}
```

Fig. 3. Ingreso de Datos y Operaciones Aritméticas

D. Código

Para la realización de la actividad se desarrollaron tres programas utilizando el lenguaje de programación C. Cada código fue elaborado y ejecutado con el propósito de aplicar los conceptos estudiados sobre tipos de datos, variables, valores booleanos, operadores aritméticos y estructuras condicionales.

Programa 1: Valores Booleanos

En el primer programa se trabajó con valores booleanos, utilizando las variables true y false para representar estados lógicos. Asimismo, se utilizaron instrucciones de salida para mostrar los valores obtenidos durante la ejecución del programa.

```
GNU nano 8.7.1 tareab3a.c
#include <stdio.h>
#include <stdbool.h>
#include <stdlib.h>

En C, el tipo de datos bool no es un tipo integrado, es una macro definida en el archivo de encabezado stdbool.h

int main() {
    bool x = true;
    bool y = false;
    system("clear");
    printf("x %d\n", x);
    printf("y %d\n", y);
    return 0;
}
```

Fig. 1.Creación del Primer Programa

Programa 2: Tipos de datos

En el segundo programa se declararon variables utilizando diferentes tipos de datos, como enteros, caracteres y números decimales. Posteriormente, se utilizaron formatos de salida para mostrar correctamente los valores almacenados en cada variable.

```
GNU nano 8.7.1 tareab3b.c
#include <stdio.h>
#include <stdbool.h>
#include <stdlib.h>

En C, el tipo de datos bool no es un tipo integrado, es una macro definida en el archivo de encabezado stdbool.h

int main() {
    int i = 11;
    int j = 12;
    char c = 'A';
    float n=0.791512;
    system("clear");
    printf("x %3d %c %3f\n",i,j,c,n);
    return 0;
}
```

Fig. 2.Declaración de Diferentes Tipos de Datos

Programa 3: Operaciones aritméticas

En el tercer programa se solicitaron dos números al usuario para realizar operaciones de suma, resta, multiplicación y división. Además, se implementó una estructura condicional para comprobar que no se intentara realizar una división entre cero.

```
GNU nano 8.7.1
#include <stdio.h>
int main() {
    double num1, num2;

    printf("Ingrese el primer numero: no un caracter. ");
    scanf("%lf", &num1);

    printf("Ingrese el segundo numero: no un caracter. ");
    scanf("%lf", &num2);

    double suma = num1 + num2;
    double resta = num1 - num2;
    double multiplicacion = num1 * num2;

    printf("\nResultados: \n");
    printf("Suma: %.4lf\n", suma);
    printf("Resta: %.4lf\n", resta);
    printf("Multiplicacion: %.4lf\n", multiplicacion);

    if (num2 == 0) {
        printf("Error: No se puede dividir entre cero. \n");
    } else { double division = num1 / num2;
        printf("Division: %.4lf\n", division);
    }

    return 0;
}
```

Fig. 3. Ingreso de Datos y Operaciones Aritméticas

Enlace al Repositorio de Git Hub:

<https://github.com/mariaferorellanam/Introducci-n-a-la-Programaci-n-de-Computadoras>

E. Resultados

Al finalizar la elaboración de los programas, se realizaron las respectivas ejecuciones para comprobar su funcionamiento y verificar que los resultados obtenidos fueran los esperados.

1. Resultado del Programa de Valores Booleanos

El primer programa se ejecutó correctamente y permitió visualizar en la terminal los valores correspondientes a las variables booleanas declaradas, comprobando el funcionamiento de los valores `true` y `false`.

```
1
0
maferom@ubuntulinux:~/0769$
```

Fig. 4.Resultados de Creación del Primer Programa

2. Resultado del Programa de Tipos de Datos

El segundo programa mostró correctamente en la terminal los valores almacenados en las variables de diferentes tipos de datos. Los resultados permitieron comprobar que cada valor fue presentado utilizando el formato correspondiente.

```
b 12 A 40.792
maferom@ubuntulinux:~/0769$ _
```

Fig. 5.Resultado de Declaración de Diferentes Tipos de Datos

3. Resultado del Programa de Operaciones Aritméticas

El tercer programa recibió los valores ingresados por el usuario y realizó correctamente las operaciones de suma, resta, multiplicación y división. Además, se comprobó la validación implementada para evitar una división entre cero.

```
Ingrese el primer numero: no un caracter. 15
Ingrese el segundo numero: no un caracter. 12

Resultados:
Suma: 27.0000
Resta: 3.0000
Multiplicacion: 180.0000
Division: 1.2500
maferom@ubuntulinux:~/0769$ _
```

Fig. 6. Resultado de Ingreso de Datos y Operaciones Aritméticas

En general, los tres programas se ejecutaron de manera correcta, permitiendo comprobar la aplicación de los conceptos básicos de programación en lenguaje C trabajados durante la actividad.

F. Conclusiones

Durante el desarrollo de la actividad se logró aplicar los conocimientos básicos del lenguaje C mediante la elaboración de tres programas. En el primero se trabajó con valores

booleanos utilizando `true` y `false`; en el segundo se aplicaron diferentes tipos de datos, como `int`, `char` y `float`, junto con sus respectivos formatos de salida. Finalmente, en el tercer programa se utilizaron variables, entrada de datos mediante `scanf()`, operadores aritméticos para realizar suma, resta, multiplicación y división, y estructuras condicionales `if` y `else` para controlar la división entre cero. Con la ejecución de los tres programas se comprobó el funcionamiento de los conceptos y herramientas utilizados durante la actividad.

REFERENCIAS

- [1] [1] ISO/IEC, ISO/IEC 9899:2018: Information technology — Programming languages — C, 4th ed. Geneva, Switzerland: International Organization for Standardization, 2018.
- [2] [2] Free Software Foundation, Using the GNU Compiler Collection (GCC), GNU Project. [En línea]. Disponible en: [Manual de GCC](#)
- [3] [3] Free Software Foundation, The GNU C Library: Formatted Output, GNU Project. [En línea]. Disponible en: [Biblioteca GNU C — Salida formateada](#)
- [4] [4] Free Software Foundation, The GNU C Library: Input/Output on Streams, GNU Project. [En línea]. Disponible en: [Biblioteca GNU C — Entrada y salida](#)
- [5] [5] The Open Group, stdbool.h — Boolean type and values, Base Definitions, IEEE Std 1003.1. [En línea]. Disponible en: [Referencia de stdbool.h](#)
- [6] [6] cppreference.com, Type — C language reference. [En línea]. Disponible en: [Referencia de tipos de datos en C](#)
- [7] [7] cppreference.com, Arithmetic operators — C language reference. [En línea]. Disponible en: [Referencia de operadores aritméticos en C](#)
- [8] [8] cppreference.com, if statement — C language reference. [En línea]. Disponible en: [Referencia de la estructura if en C](#)